

Building Element

The Advanced Building System is a modular building system and the structures are created and built up from the building elements. The building elements are the actual module objects placed by the Advanced Building System. These elements are represented by the `ABS_BuildingElement` script.

An element can be a building piece of a structure like a wall or a floor element. It can be a door or a window. It can be furniture like a table or a chest. It can be a tree or a turret. It just depends on you and your project.

Setup

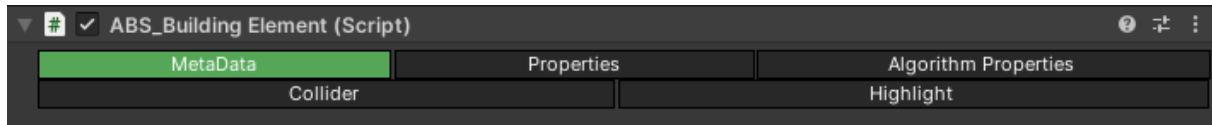
You need to make your own building elements for your game. You can do it by yourself or you can use the Advanced Building System's custom tool the `ABS_BuildingElementCreator` which will be discussed in the Tools documentation.

Every `BuildingElement` should have the following components:

- Box Collider
- `BuildingElement` (Script)

Note that the `SnapPointBased` building algorithm does not require the Box Collider but it needs at least some kind of collider. More information later in the Collider section.

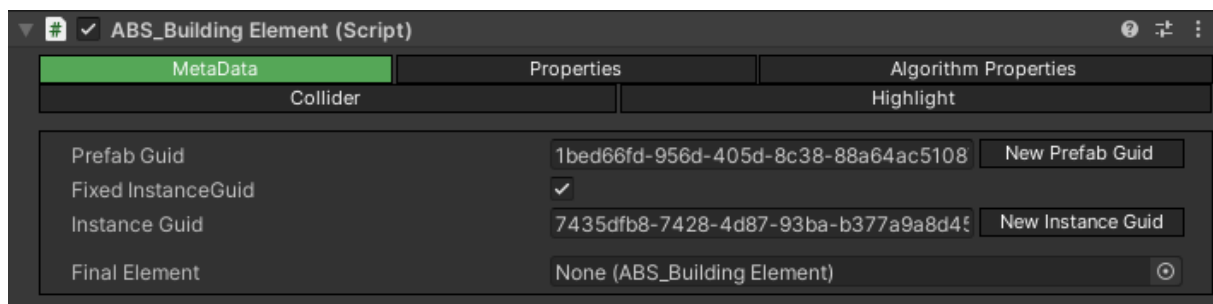
Properties Of BuildingElement



The UI of the ABS_BuildingElement script is a tab based interface. In the following section the following tabs will be discussed:

- Metadata
- Properties
- Algorithm Properties
- Collider
- Highlight

Metadata



The **Prefab Guid** parameter is used for identifying a BuildingElement. The guid is important to be unique for every different element.

Note that the Prefab Guid is for the prefab of the BuildingElement not for the actual instance of the element. It is unique for the prefab but not for the instances.

The “New Prefab Guid” button can be used for generating new unique guid.

For example if you copy a BuildingElement the properties of the script will be the same just like the Prefab Guid. So the algorithm will think that these are the same BuildingElement based on that guid. To fix this issue if you copied the BuildingElement regenerate the guid.

Instance Guid: The InstanceGuid is meant to be unique for every single instance of any ABS_BuildingElement. This is used to identify the actual instance of the elements, not the element’s prefab. It is used for the History, Persistency features and makes the handling of the elements easier from the through the tracker (API) if every instance can be identified with a Guid. The Guid is generated in the construction of the BuildingElement or it is created and overridden by the algorithm if the element is recreated by the Persistency or the History features to ensure that the destroyed (undo) and later replaced (redu) element has the same guid as the original one.

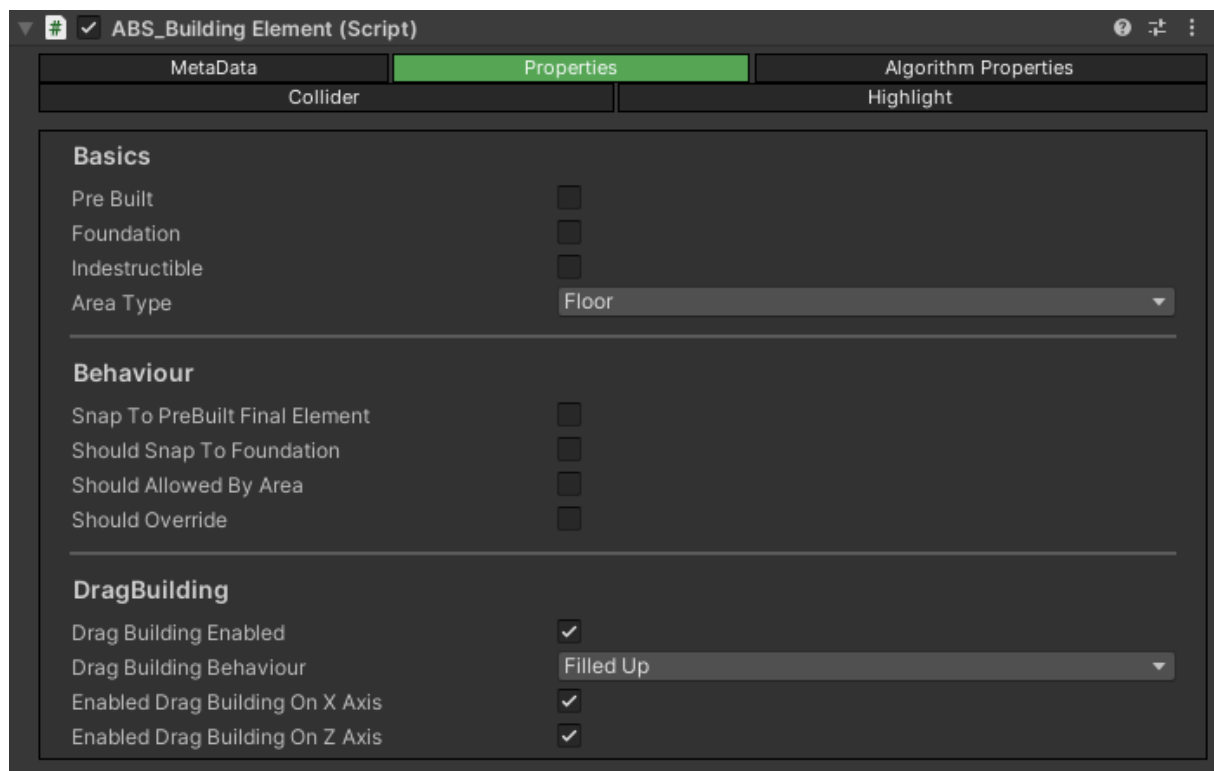
Fixed InstanceGuid is a boolean and if it is true then the InstanceGuid can be overridden by hand from the Unity’s inspector view for ensure that the guid is predictable what can be used for example for the elements what already placed by the developers and it make it easier to identify them during runtime.

Recommended to set the Fixed InstanceGuid for every element that is placed by the developers and not by the algorithm.

The **Final Element** is a BuildingElement that will be built instead of the actual BuildingElement. It is working like that during the build process the algorithm will use the actual element but when the element is finalized so the player releases the button to build the algorithm and replace the current element to the final element.

In detail the algorithm checks at the end of the build process if the BuildingElement has a Final Element (it is not null) then destroys the current element and instantiates one from the Final Element and places it right in the actual element's position. It is working in simple building and drag building scenarios too. The main idea behind this feature is that you can use another element during the build process. Like you can use a half constructed wall but when you build it the whole wall will be placed. Or you can use a generic element like a cube during the build process and place a table instead of that.

Properties



The **PreBuilt** is a boolean property which means that the BuildingElement should be handled as a PreBuilt element or not. The PreBuilt Element is explained in detail later in the documentation.

Foundation is also a boolean. The algorithms' has the "Use Foundation Logic" property. If the algorithms' property is true then a BuildingElement can be built only if it is a foundation or if it is snapping to another already built element.

Indestructible is a boolean value. If it is true then the destroy functionality will ignore the BuildingElement so it can not be destroyed by the BuildingManager. Also it will not be highlighted.

Note that this ignore means that the element is handled like the raycast hit something that isn't a BuildingElement. If you are using the Destroy feature with Timer and the "Cut Timer On Look Away" enabled the timer will be stopped when the player looks to an Indestructible element because it is handled like it is not a BuildingElement.

AreaType is used for Area features that will be explained in detail later. With this enum you can set the type of the BuildingElement for the Area feature.

Snap To Pre Built Final Element: When you are building the current element the algorithm will try to snap your element into a Pre Built element if it is nearby. It is only working when the element that was used to build is the same as the Pre Built element. So if you are using a Final Element and you are placing a half constructed wall but the final element is a whole wall the algorithm will snap your element only into the half constructed pre built wall and it can not snap into the whole pre built wall even if it will build a whole wall at the end of the process instead of the half wall. If the "Snap To Pre Built Final Element" boolean is true the algorithm allows you to snap your element into a pre-built element from your final element.

Should Snap To Foundation is a boolean value and if it is true then the element can be only built when it is snapping to a Foundation element.

Should Allowed By Area means that if it is true the BuildingManager will block the building of that element until any of the BuildingArea doesn't allow it.

The **Should Override** means that if it is true the BuildingManager will block the building of that element until it is overriding another element with the Override Element feature.

Note that this feature doesn't allow the element to snap to an PreBuilt element because that scenario is not overriding.

If the **Drag Building Enabled** then the element can be built with the Drag Building functionality also the Drag Building properties are only available on the GUI if it is set.

The [Drag Building Behaviour](#) will be discussed later in this document.

If the **Enabled Drag Building On X Axis** is set to false then the algorithm does not allow the drag building of this element on the X axis. The default is true for allowing it.

The **Enabled Drag Building On Z Axis** is work in the same way as the X Axis just on the Z axis.

PreBuilt BuildingElement

Sometimes that can be useful if some building element is already in the scene to show to the player where that element should be placed.

So what is a PreBuiltBuildingElement? The PreBuiltBuildingElements are basic BuildingElements but

- it can not collide with the player
- it has a special highlight material
- it handled specially by the AdvancedGridBuilding
 - The BuildingElements can not snap into its position just if it is the same element.
 - The BuildingElements can not snap to a position if every neighboring BuildingElement is PreBuilt.
 - The DragBuilding should handle the PreBuilt elements.
 - If a PreBuilt element from the same BuildingElement is nearby the algorithm will prioritize its position.
 - The BuildingElements can not be validated by neighbor PreBuiltBuildingElements.
- the PreBuiltBuildingElement should be destroyed if a right element is snapped into its position.

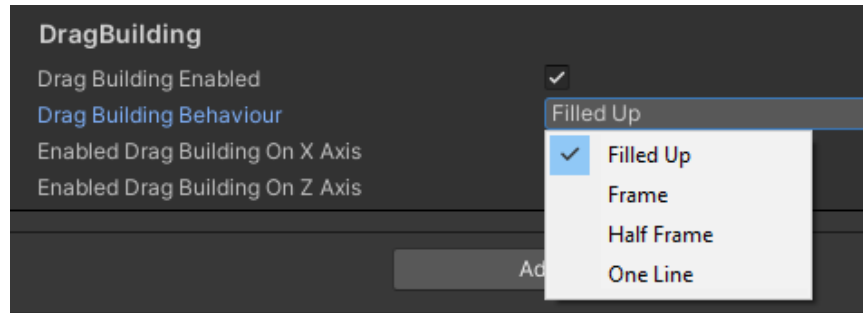
The main goal is that the developers are able to make such BuildingElements that can not be collided by the player because they are “not placed yet”. When the player is building something and a PreBuilt element with the same type is nearby the element should be snapped right into its position.

DragBuilding Behaviour

You have the following options:

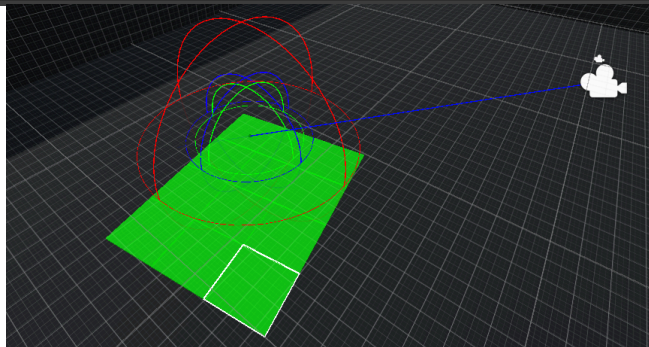
1. Filled Up
2. Frame
3. Half Frame
4. One Line

With these options you can change the shape of the drag building results.



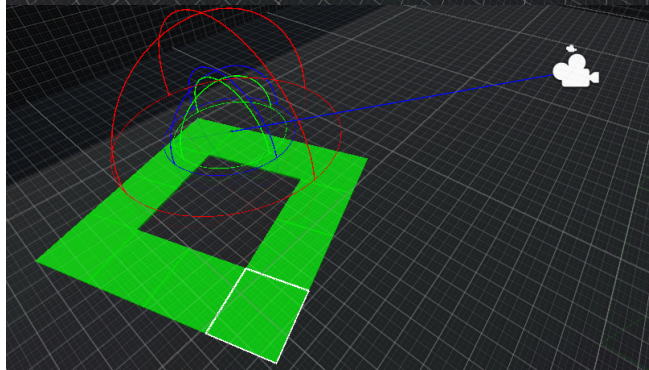
Filled Up:

With the Filled Up setup the algorithm will fill up the whole area. So every line and every row will be filled up with elements.



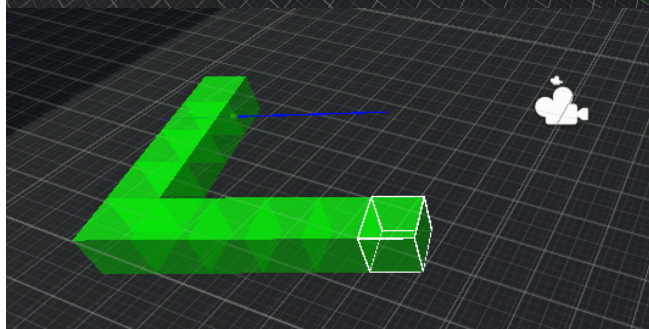
Frame:

With the Frame Setup the algorithm will place the elements only on the edge of the area. The inner elements are not placed.



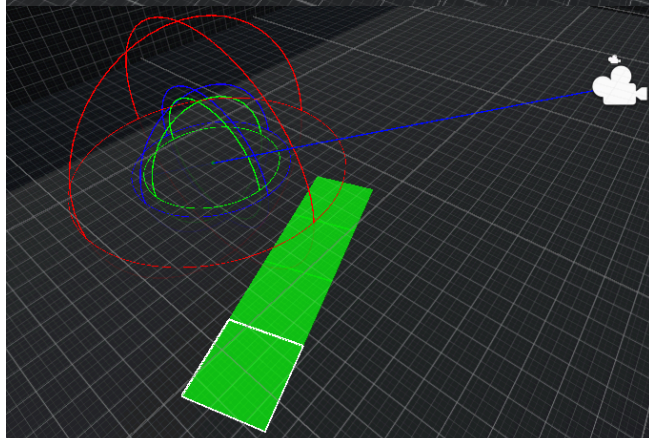
Half Frame:

It is basically the same mechanism as the Frame, it just only places the elements on the longer side.



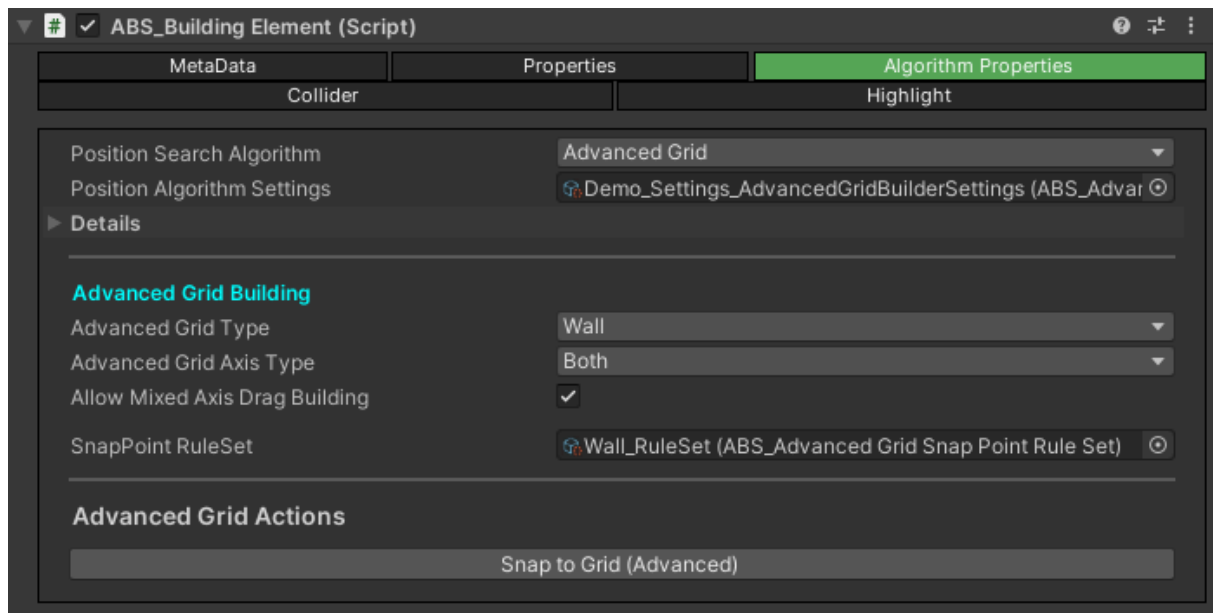
One Line:

In case of One Line only the first row or the first column will be placed. Which is longer.



What is the difference from the OneLine setup and the limit of an axis? If you limit one of the axes then the BuildingManager will build only on the other axis. But with this setup you can build a direct line on any axes. The algorithm will check the raycast's results and it will build the line on the X or Z axes which one is nearest to the raycast's hit point (the longer one). So with OneLine you can get a direct line on any of the axes.

Algorithm properties



Position Search Algorithm

Every BuildingElement needs an algorithm setting. The elements will be built based on that setting. First the algorithm type should be selected.

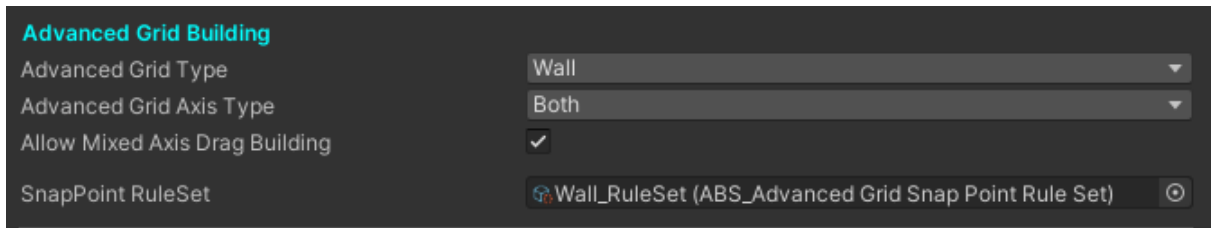
The following algorithms are available:

- Free
- Basic Grid
- Advanced Grid
- Snap Point Based

More information about the algorithms are in the BuildingAlgorithm documentation.

Position Algorithm Settings is the algorithm settings itself (ScriptableObject).

Advanced Grid Building

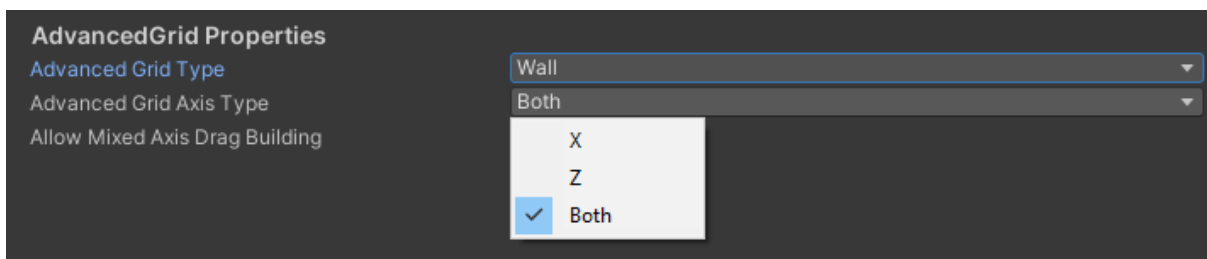


The BuildingElement has some specific properties for the AdvancedGrid Building algorithm.

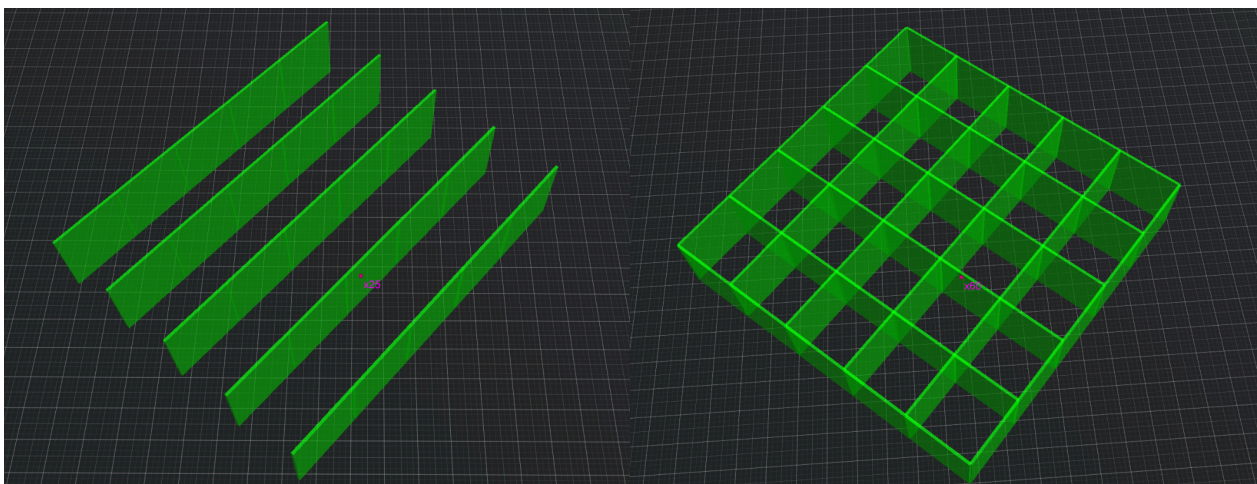
First the Advanced Grid Type so what kind of element is the element:

- Floor
- Wall
- Horizontal Edge
- Vertical Edge
- Corner
- Center

In case of Wall or Horizontal Edge the Advanced Grid Axis Type should be set too. The developers can force the building of this element to only one axis.



The Allow Mixed Drag Building is also available for the Wall and the Horizontal Edge. If this is true then The algorithm will use both axes during the Drag Building.

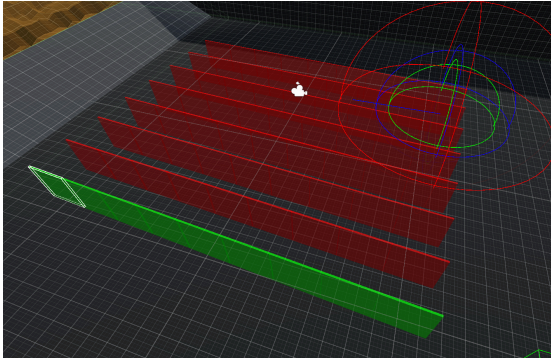


Drag Building Behaviours + Mixed Axis

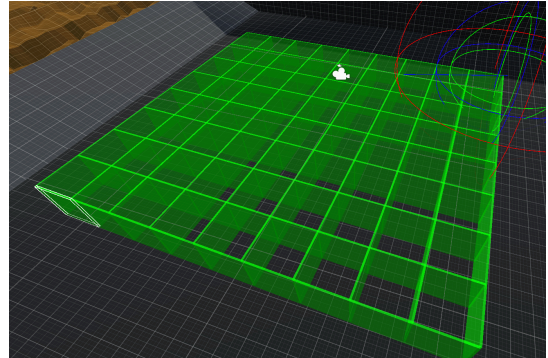
In this chapter you can see how the different Drag Building Behaviours work with mixed axis in case of Advanced Grid Building and a wall or a horizontal edge.

Note: in this example some elements on the Z axis don't touch the building so they are blocked. In other cases like building on a big platform they would be allowed.

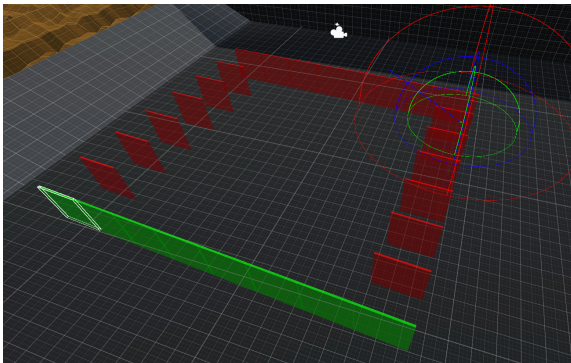
Filled Up



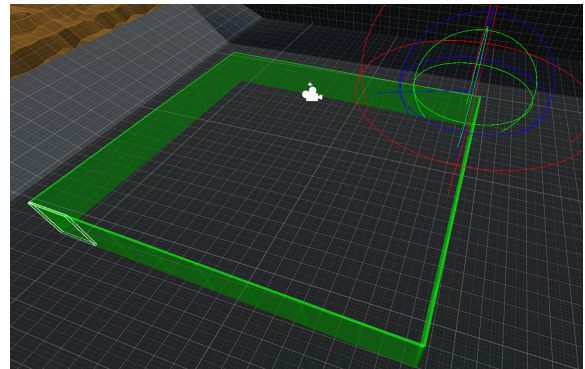
Filled Up + Mixed Axis



Frame

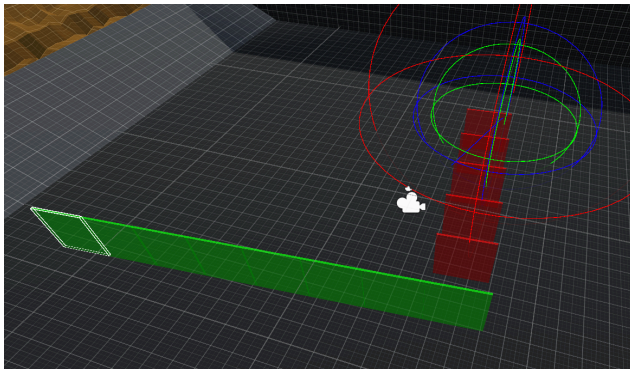


Frame + Mixed Axis

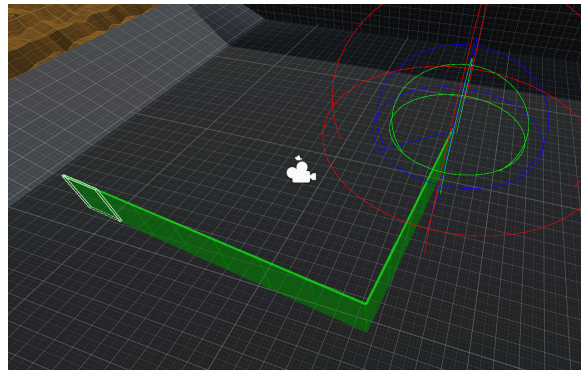


Half Frame

(X is the longer Side)

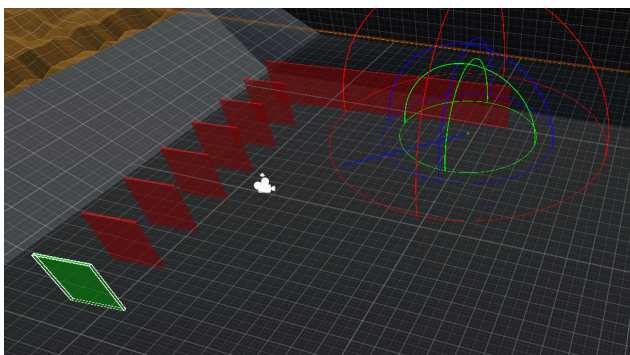


Half Frame + Mixed Axis

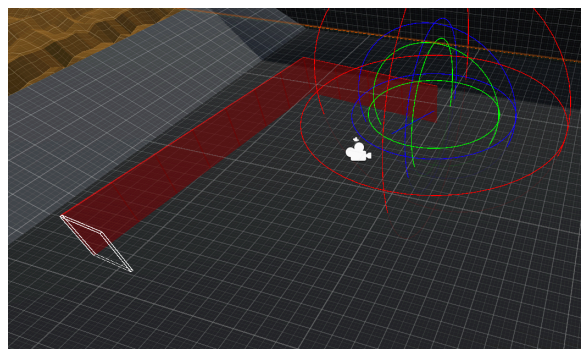


Half Frame

(Z is the longer Side)

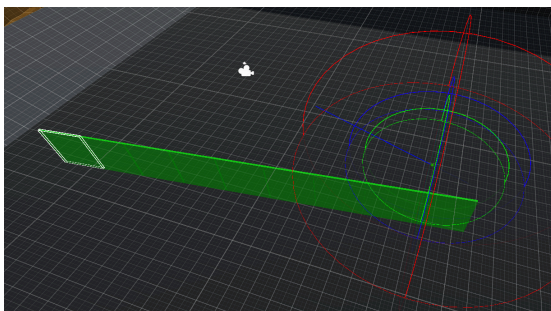


Half Frame + Mixed Axis

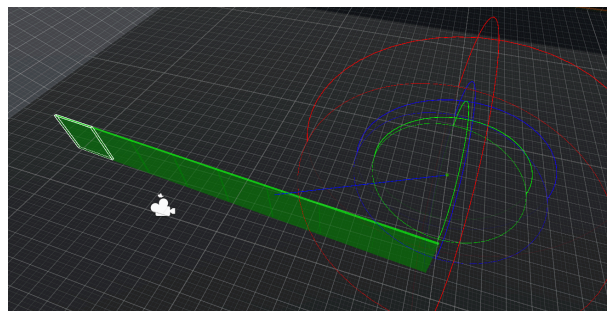


OneLine

(X is the longer Side)

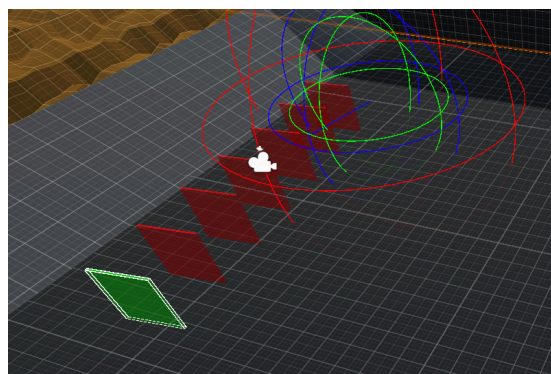


OneLine + Mixed Axis

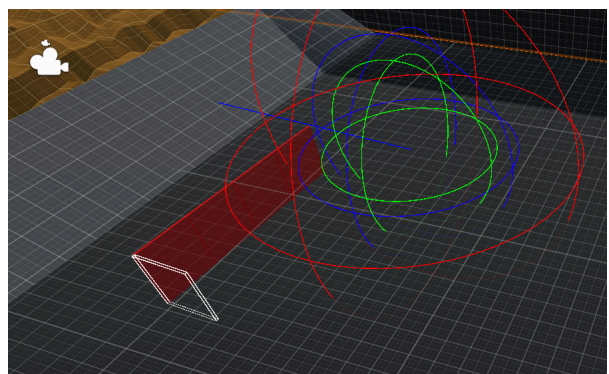


OneLine

(Z is the longer Side)

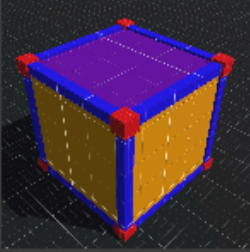


OneLine + Mixed Axis



Advanced Grid Snap Point Rule Set

The AdvancedGrid algorithm can use a rule set for every element. It can be null or it can be unique for every element. The ruleset is a ScriptableObject (ABS_AdvancedGridSnapPointRuleSet) which contains the rules like blocking or denying a snap point. For more information check the BuildingAlgorithms documentation.



AdvancedGridSnapPointRuleSet

Checkout

Target Element Type : Center

▶ Floor

▶ Wall

▶ EdgeHorizontal

▶ EdgeVertical

▼ Corner

Position (0) : (0.50, 0.50, 0.50)	Permission : Allow
Position (1) : (0.50, 0.50, -0.50)	Permission : Block
Position (2) : (-0.50, 0.50, 0.50)	Permission : Allow
Position (3) : (-0.50, 0.50, -0.50)	Permission : Block
Position (4) : (0.50, -0.50, 0.50)	Permission : Allow
Position (5) : (0.50, -0.50, -0.50)	Permission : Allow
Position (6) : (-0.50, -0.50, 0.50)	Permission : Allow
Position (7) : (-0.50, -0.50, -0.50)	Permission : Allow

▼ Center

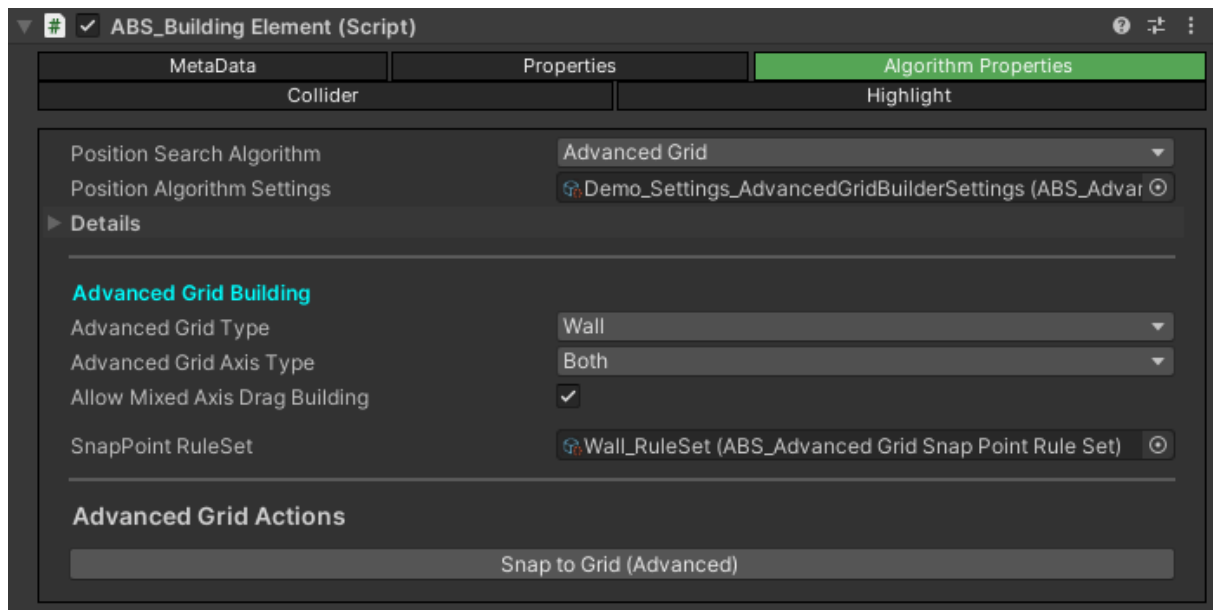
Position (0) : (1.00, 0.00, 0.00)	Permission : Allow
Position (1) : (-1.00, 0.00, 0.00)	Permission : Allow
Position (2) : (0.00, 0.00, 1.00)	Permission : Block
Position (3) : (0.00, 0.00, -1.00)	Permission : Block
Position (4) : (0.00, 1.00, 0.00)	Permission : Block
Position (5) : (0.00, -1.00, 0.00)	Permission : Block

Edit

Snap to Grid

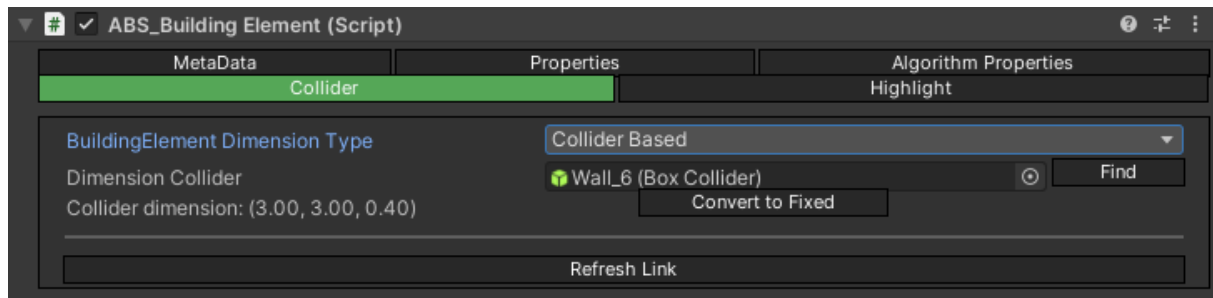
“Snap to Grid (Advanced)” and “Snap to Grid (Basic)” Buttons are the only available algorithm specific options of the ASB_BuildingElements. These 2 buttons can be used during the scene building process when the developer is placing the building elements by hand and he/she wants to ensure that the elements are snapping to the grid. Just place the element to the scene and press the button. It will modify its transform to snap into the grid. Note that this feature is only working if the BuildingElement's parent object has the proper type of Building script as component.

In case of multi-object editing (multiple elements are selected) the user interface will show a list about the elements that will be affected by the “Snap to Grid” feature.



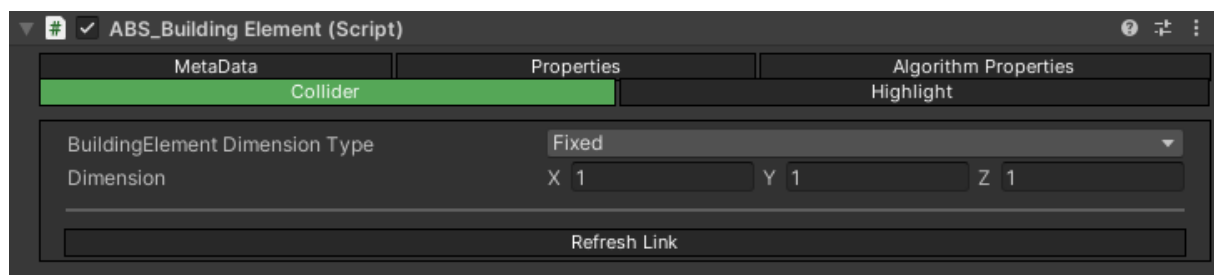
Collider

The collider is a crucial point of the algorithm. The collider is used for validation checks, raycast and also the building element size the dimension is calculated from the collider.



The algorithms are calculating only with box colliders. Sometimes a box collider can not be used for the element so the dimension can be set by hand too.

If the Collider Based used the box collider on the element can be added by hand or found by the element's script too. If a collider is already added it can be converted to Fixed setup. If the convert is triggered, then the box collider's size will be used as the Dimension.



Refresh Link

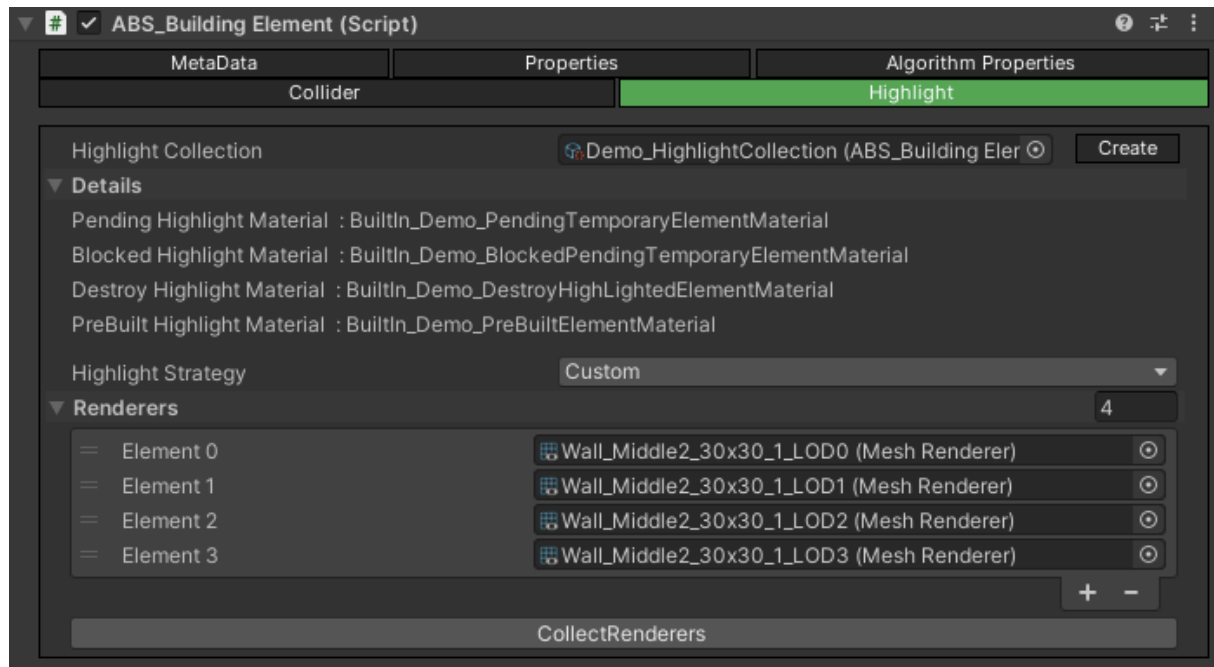
If the element has a child object that has a bigger collider or it has an exactly same collider as the element's collider then the algorithm can not find the element with a raycast during the destroy process.

With this refresh button the Advanced Building System will try to automatically check and fix the linkage of the element.

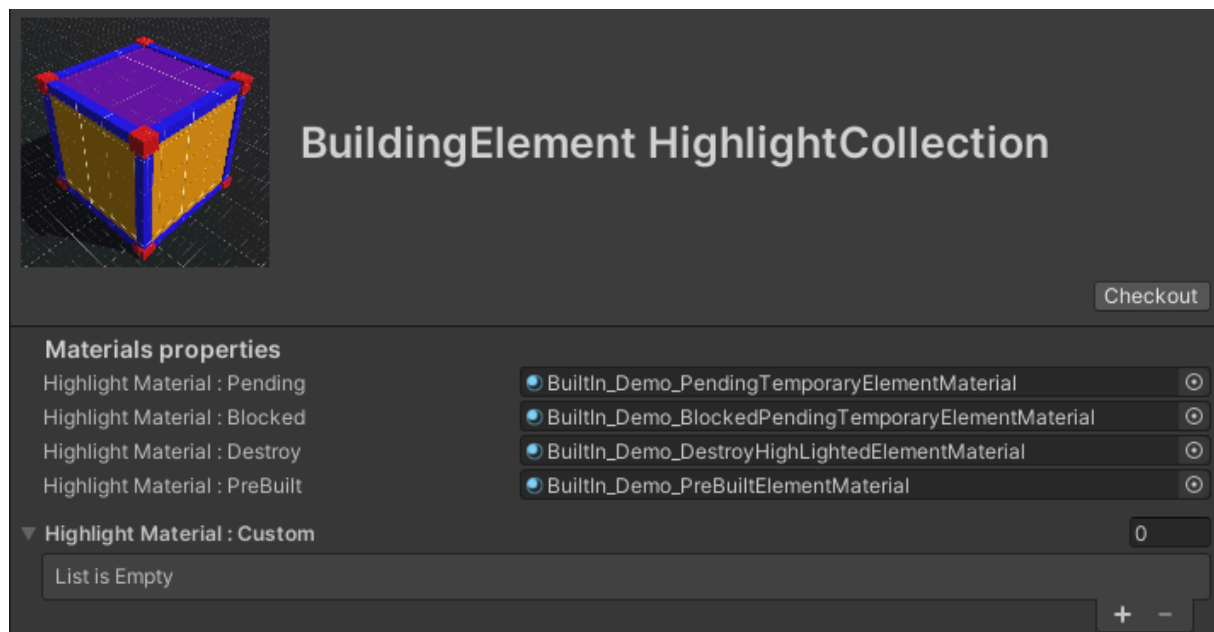
For more information check the BuildingElementLink object's description in the Tools documentation.

Highlight

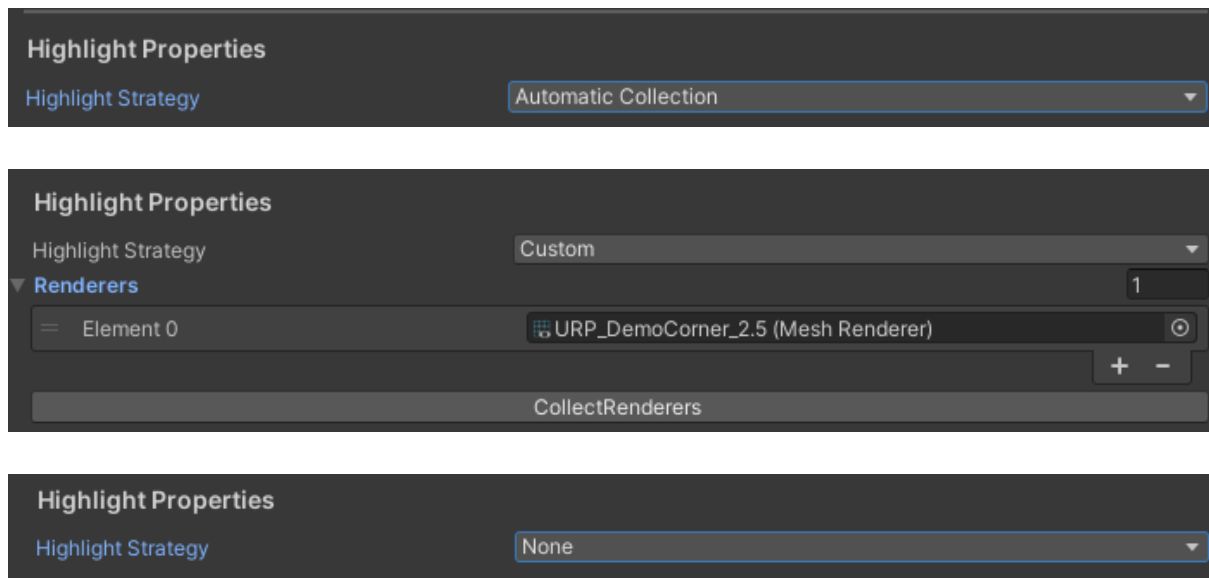
The BuildingManager uses a highlight feature that will change or modify the materials of the renderers of the ABS_BuildingElement. It is used to highlight the different states of the BuildingElements like blocked or pre built etc.



The Highlight Collection is waiting for an instance of an [ABS_HighlightCollection](#) ScriptableObject which is basically a collection of the highlight materials for the building elements.



You can select from the following 3 behavior:



None: There will be no highlighting. The Building Element will get the event and change its states but doesn't change the materials at all. If you don't want the highlight feature just set this property to None.

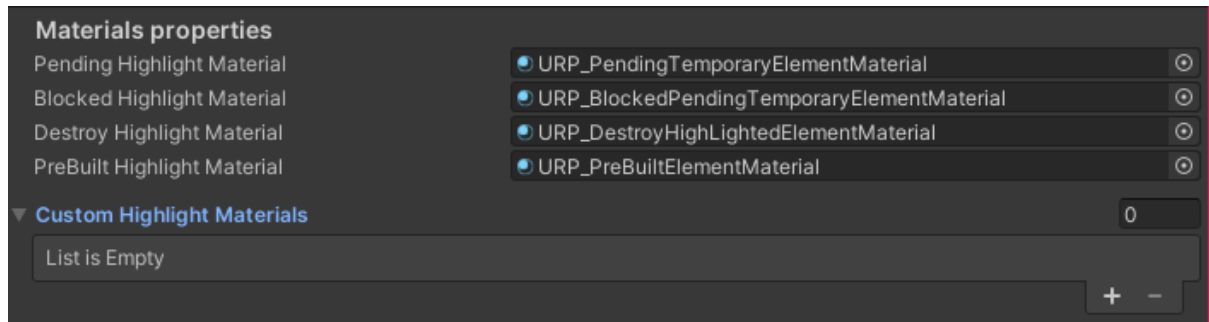
Automatic Collection: The BuildingElement will collect the rendered from its game object and its childs and use the highlight logic so it will modify the materials of the renderer. Note that this will collect every renderer and change these materials.

Custom: It is basically the same as the Automatic Collection. The Building Element will use the highlight feature and it will change the material of the rendere just in this time you can set by hand which renderers should be used. To make it easier to collect the renderers of the GameObject the CollectRenderers button can be used for automatically collecting the renderers just like the Automatic Collection. Just in this time you can modify the list of the elements by hand.

Highlight Collection

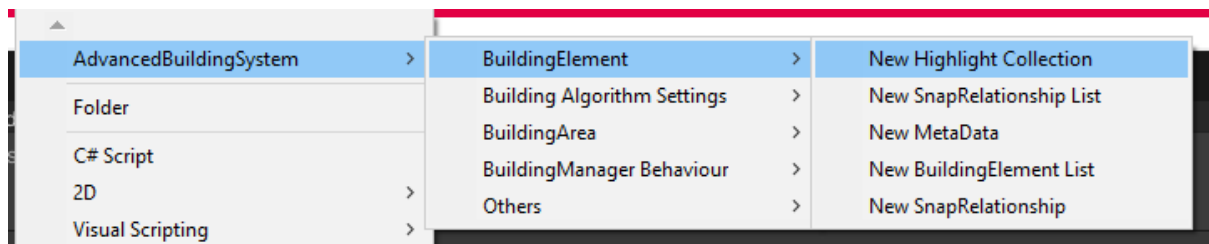
The HighlightCollection is just a list of the materials that should be used for highlighting the building elements in different states. And also a list of different materials for custom highlighting the elements.

This custom highlight feature can be done by calling the **SetHighlightMaterial** function on the ABS_BuildingElement script and providing the index of the material from the “Custom Highlight Materials” list.

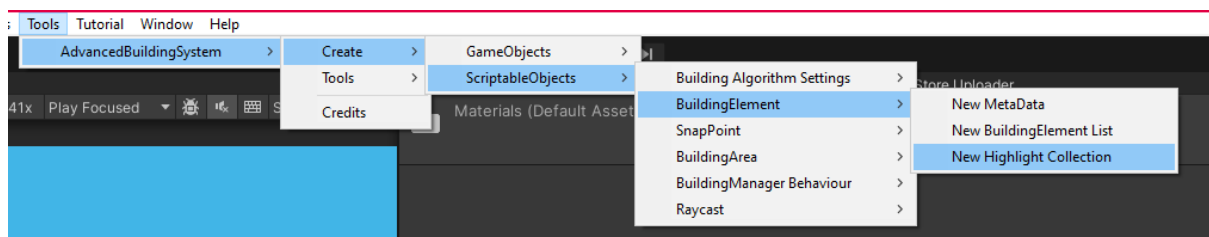


The Highlight Collection can be created just like every other scriptable object used by the Building Manager.

From the project:



From the Menu:



ABS_IBuildingElementExternalInterface

The ABS_BuildingElement has a lot of functions that are used by the algorithm but some of them are important to the developers who are using this asset. To separate the functions that are meant to be used by the users of the asset the BuildingElement is implementing the IBuildingElementExternalInterface interface. This interface is a collection of public functions that are safe to use by the developers. Do not recommend using any other function on the BuildingElement because it can easily break the logic of the Advanced Building System.

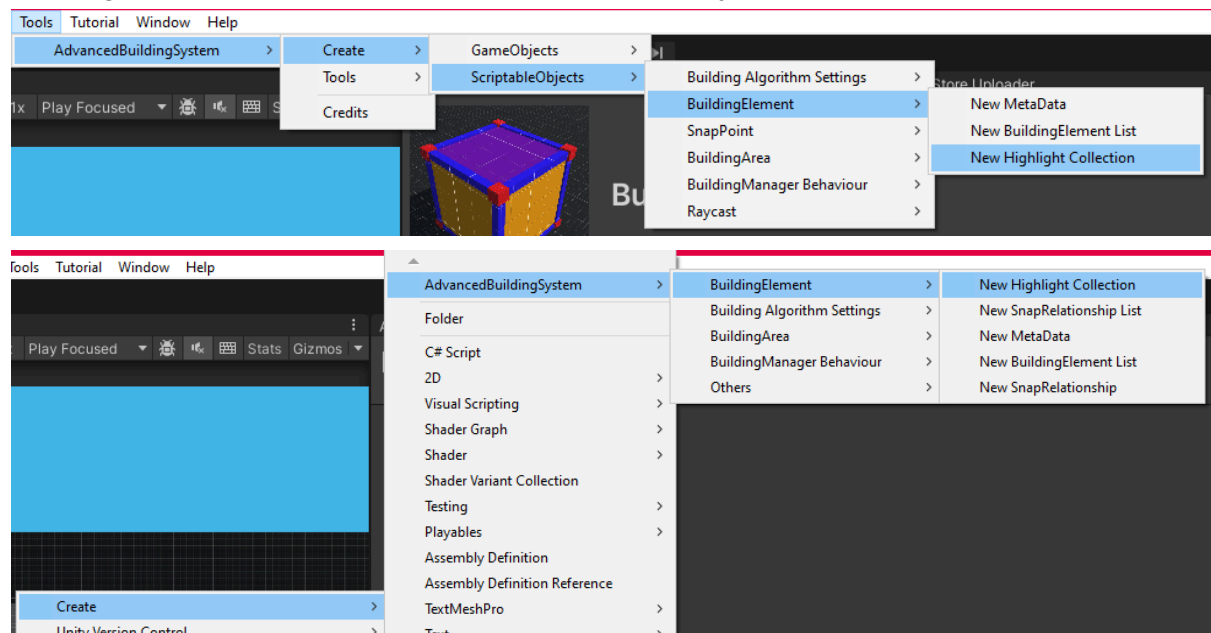
BuildingElementList

Currently the BuildingElementList is just a list of BuildingElements. It should be provided to the BuildingManager. The manager will use it to get the BuildingElement GameObjects to instantiate the elements.

When the BuildingManager is activated through the IBuildingManagerExternalInterface interface an index of the BuildingElement from the list should be provided to the Manager. The list can be changed during the game using the **SetBuildingElementList** function of the earlier mentioned interface.

This list is used by the History feature. If an element is not contained by the list the undo/redo features will fail.

Creating the List can be done from the Menu or the project Create option:

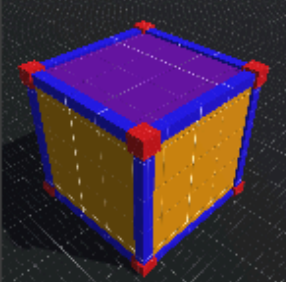


The secondary feature of the BuildingElementList is a validation action that can be triggered on the UI. The validation will try to check some issues and give feedback for the developers.

The following will be checked:

- Null element in the list. (None or Missing elements)
- Duplicated elements (Same PrefabGuid)
- Blocked Drag Building (Both axes are blocked for building. X and Z)
- Null Dimension Collider (null collider with collider based setup so the dimension)

In the following image you can see a failed validation that is trying to show the possible issues.



BuildingElementList

Checkout

▼ BuildingElements

9

Element 0	URP_AdvancedGridDemo_Wall_2.5 (Building Elem
Element 1	None (Building Element)
Element 2	None (Building Element)
Element 3	URP_AdvancedGridDemo_Wall_2.5 (Building Elem
Element 4	URP_AdvancedGridDemo_Wall_2.5 1 (Building Elei
Element 5	URP_AdvancedGridDemo_WallDoor_2.5 (Building
Element 6	Missing (Building Element)
Element 7	Missing (Building Element)
Element 8	URP_AdvancedGridDemo_Wall_2.5 2 (Building Ele

+ -

Validate

! Invalid Object at : 1, 2, 6, 7

! The following elements are sharing the same PrefabGuid!

PrefabGuid: e4ea9504-6c96-4160-ab00-fb27582643bb

0		URP_AdvancedGridDemo_Wall_2.5
3		URP_AdvancedGridDemo_Wall_2.5
4		URP_AdvancedGridDemo_Wall_2.5 1
5		URP_AdvancedGridDemo_WallDoor_2.5

! The following elements hasn't MetaData

8		URP_AdvancedGridDemo_Wall_2.5 2
---	---	---------------------------------